MongoDB en el Sistema General de Ingresos: documentos para datos sin forma de tabla

Equipo 3 · Heidi, Scarlet, Damian

Resumen

El Sistema General de Ingresos guarda su información en MySQL, un motor relacional que funciona bien para datos con forma de fila. Sin embargo, los trámites que administra no tienen todos la misma estructura: una licencia de funcionamiento y una constancia de no adeudo comparten folio y fecha, pero no comparten campos específicos. Este reporte analiza MongoDB como almacén complementario para ese tipo de datos. Se describe su modelo de documentos, las piezas con las que se trabaja, la forma de operarlo desde una API y un ejemplo concreto con dos trámites en una misma colección. Se concluye que MongoDB no sustituye a MySQL ni elimina el diseño del esquema: lo traslada a la aplicación, y esa decisión es la que determina si la herramienta ayuda o estorba.

1. Introducción

En el curso se construyó una API sobre MySQL, con Docker para levantar los servicios, GraphQL como contrato de consulta y un token para controlar el acceso. Ese conjunto funciona porque los datos del sistema tienen una estructura estable: usuarios, pagos, adeudos y permisos se pueden expresar como filas con columnas conocidas.

El problema aparece con los trámites. Cada tipo de trámite tiene sus propios campos. Si se modelan en una sola tabla, esta acumula columnas que solo aplican a algunos registros y termina llena de valores `NULL`. Si se modelan con una tabla por tipo, cada trámite nuevo obliga a crear una tabla, modificar la API y escribir otra consulta. En ambos casos, el esquema de la base de datos se convierte en el obstáculo: hay que migrarlo antes de poder guardar el dato.

La pregunta que guía este reporte es dónde guardar información cuya forma cambia con facilidad. La respuesta que se defiende es usar MongoDB como almacén complementario para documentos, manteniendo MySQL como fuente de verdad para las operaciones relacionales y transaccionales.

2. Qué es MongoDB

MongoDB es una base de datos orientada a documentos. La unidad de almacenamiento no es la fila, sino el documento: un objeto en formato JSON que se guarda serializado como BSON. Los documentos se agrupan en colecciones y las colecciones viven dentro de una base de datos. Una colección se parece a una tabla en que agrupa registros del mismo tema, pero se diferencia en que no impone una lista fija de columnas: dos documentos de la misma colección pueden tener campos distintos.

El esquema no desaparece; cambia de lugar. En MySQL lo define la base de datos y el servidor rechaza lo que no cumple. En MongoDB el servidor acepta el documento y la aplicación decide qué campos son obligatorios, cuáles son opcionales y cómo se validan. Esto da flexibilidad para agregar tipos de trámite, pero traslada al equipo la responsabilidad de diseñar y mantener ese contrato.

Las piezas con las que se trabaja son cuatro:

- **Servidor** (`mongod`): proceso que almacena los documentos y atiende las conexiones en el puerto 27017.
- **Cliente**: biblioteca dentro de la API. En Python se usa `pymongo`; en Node, `mongoose`.
- **Consola** (`mongosh`): shell interactivo para insertar, consultar y revisar índices.
- **Atlas**: servicio administrado en la nube, útil cuando no se quiere operar el servidor.

Además de las piezas, existen dos mecanismos que determinan el rendimiento y la confiabilidad: los índices, que evitan recorrer toda la colección en cada consulta, y las réplicas, que permiten tolerar la caída de un servidor.

3. Cómo se usa

El uso desde una API sigue cuatro pasos.

1. **Elegir base y colección.** Por ejemplo, base `ingresos` y colección `tramites`.
2. **Armar el documento.** Un objeto con los campos comunes (`tipo`, `folio`, `fecha`, `estado`) y un objeto `datos` con los campos propios del trámite.
3. **Insertarlo.** `db.tramites.insertOne(doc)`. No hay que declarar columnas ni ejecutar una migración previa.
4. **Consultarlo.** `db.tramites.find({ "datos.giro": "restaurante" })`. La consulta puede entrar a los objetos anidados y a los arreglos.

A esto se agrega el índice. Sin índice, MongoDB revisa todos los documentos de la colección; con `db.tramites.createIndex({ "datos.rfc": 1 })`, la búsqueda por ese campo es directa. El concepto es el mismo que en MySQL: el índice no cambia el resultado, cambia el costo de obtenerlo.

En el modelado hay una decisión central: **incrustar o referenciar**. Si un dato se lee siempre junto con el documento y no se repite demasiado, conviene incrustarlo, como los datos del trámite. Si el dato se comparte entre muchos documentos y cambia con frecuencia, conviene guardarlo aparte y referenciarlo. Incrustar de más duplica información; referenciar de más obliga a hacer varias consultas, que es justo lo que MongoDB busca evitar.

4. Ejemplo concreto en la API

El siguiente par de documentos vive en la misma colección. El primero corresponde a una licencia de funcionamiento; el segundo, a una constancia de no adeudo.

```
{
  "tipo": "licencia",
  "folio": "L-1042",
  "fecha": "2026-10-20",
  "datos": { "giro": "restaurante", "m2": 85 }
}

{
  "tipo": "constancia",
  "folio": "C-2087",
  "fecha": "2026-10-21",
  "datos": { "rfc": "XAXX010101000", "adeudos": 0 }
}
```

La consulta para recuperar los trámites cuyo giro sea restaurante es:

```
db.tramites.find({ "datos.giro": "restaurante" })
```

En MySQL, el mismo caso requeriría una tabla `tramites` con los campos comunes, una tabla `licencias` y otra `constancias`, además de un `JOIN` para reconstruir el trámite completo. En MongoDB, el documento se recupera entero con una sola lectura.

Desde la API, el cambio es pequeño. El endpoint sigue recibiendo la misma petición y devolviendo el mismo JSON; lo único que cambia es de dónde se lee:

```
def get_tramite(folio):
    doc = db.tramites.find_one({"folio": folio})
    if doc:
        return doc
    return mysql.query("SELECT ... FROM tramites WHERE folio = %s", folio)
```

Este patrón permite introducir MongoDB sin reescribir el sistema: primero se escribe en MySQL y se copia el documento a Mongo, o se usa Mongo solo para los trámites nuevos. La lectura intenta Mongo y, si no encuentra, cae a MySQL. Así se prueba la herramienta con riesgo controlado.

5. Comparación con lo visto en el curso

Ya lo vimos	Cómo funciona	Qué hace MongoDB
MySQL	Filas y columnas con esquema fijo; relaciones con `JOIN`	Documentos con campos variables; datos anidados
Docker	Levanta la API y la base de datos	Un contenedor más: `mongo:7`, puerto 27017
GraphQL	Define qué datos pide el cliente	No es una API: es almacenamiento interno
Token / permisos	Deciden quién puede entrar	No autentica: la API sigue validando

La diferencia de fondo es el modelo de datos. MySQL está diseñado para relacionar entidades y garantizar consistencia; MongoDB está diseñado para leer un documento completo con pocas operaciones. Ninguno es mejor en abstracto: la elección depende de si el dato se parece más a una fila o a un documento.

6. Ventajas, costos y límites

Ventajas	Costos
Un trámite se lee completo en una consulta	Duplicar lo que se repite en varios documentos
Esquema flexible para tipos nuevos	El esquema se diseña en la aplicación, no en la base
Menos migraciones (`ALTER TABLE`)	Sin `JOIN`: las relaciones se resuelven en el código
Escala horizontal con réplicas	Transacciones más limitadas entre colecciones
Consultas a campos anidados y arreglos	Otro modelo que aprender para el equipo

El costo más importante no es técnico, sino de diseño. Al no haber un esquema impuesto por el servidor, es fácil que en pocos meses existan varias versiones del mismo documento. Para evitarlo se necesita una convención escrita: qué campos son obligatorios, cómo se nombran, qué se incrusta y qué se referencia. Sin esa convención, la flexibilidad se convierte en desorden.

7. Cuándo no conviene usarla

MongoDB no es adecuada cuando el valor está en las relaciones. En el Sistema General de Ingresos, los pagos, la contabilidad y la nómina implican movimientos que afectan varias entidades a la vez y deben ser exactos; ahí el `JOIN` y las transacciones de MySQL son la herramienta correcta. Tampoco conviene cuando los reportes cruzan muchas entidades —sumar por sucursal, mes y tipo— porque esas agregaciones son más simples en SQL. Y no conviene si el esquema ya es fijo y simple: si el dato cabe en una tabla, agregar otro motor solo suma complejidad operativa.

La pregunta para decidir es concreta: **¿este dato se lee completo y cambia por bloques?** Si la respuesta es sí, MongoDB aporta. Si el dato se lee en pedazos y se relaciona con muchos otros, MySQL sigue siendo la mejor opción.

8. Conclusiones

1. **El documento sigue al dato.** MongoDB resulta útil cuando la información tiene forma de documento y se lee completa, como un trámite con sus campos variables.
2. **El esquema se diseña, no se ignora.** La flexibilidad de MongoDB traslada la responsabilidad a la aplicación; sin convenciones, se pierde consistencia.
3. **Es una pieza, no un reemplazo.** MySQL sigue siendo la fuente de verdad para pagos, contabilidad y relaciones; MongoDB se usa al lado, para lo que no tiene forma de fila.

Con estas condiciones, MongoDB permite agregar tipos de trámite sin migrar la base cada vez y sin llenar la tabla principal de columnas vacías. Usado sin diseño, en cambio, solo cambia un problema por otro.

9. Fuentes

1. MongoDB. *MongoDB Manual*. https://www.mongodb.com/docs/manual/
2. MongoDB. *Data Modeling Introduction*. https://www.mongodb.com/docs/manual/core/data-modeling-introduction/
3. Docker Hub. *mongo (official image)*. https://hub.docker.com/_/mongo